

AI Assisted Coding Assignment- 8.2

SAI SPURTHI

BELLAMKONDA

2303A51886

BATCH 30

Task 1 Test-Driven Development for Even/Odd Number Validator

• Use AI tools to first generate test cases for a function `is_even(n)` and then implement the function so that it satisfies all generated tests.

Requirements:

- Input must be an integer
- Handle zero, negative numbers, and large integers

Example Test Scenarios:

`is_even(2) → True`

`is_even(7) → False`

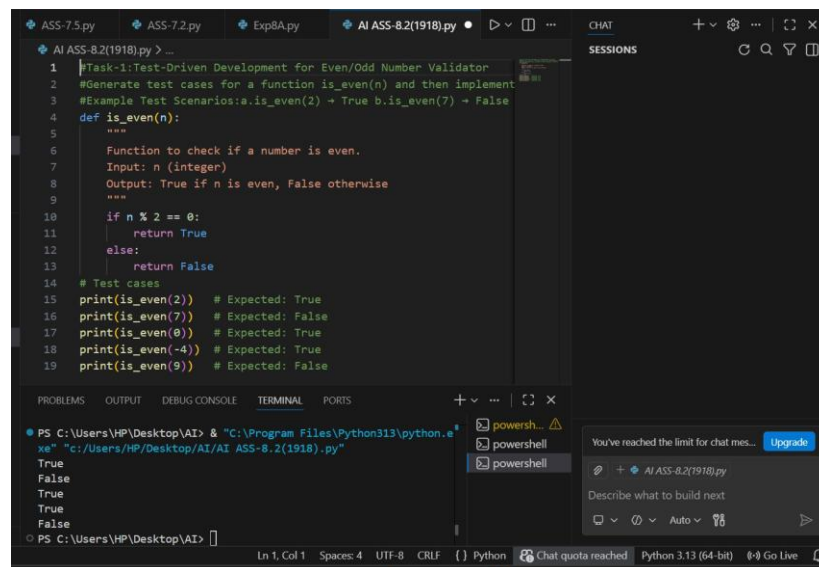
`is_even(0) → True`

`is_even(-4) → True`

`is_even(9) → False`

Expected Output 1

• A correctly implemented `is_even()` function that passes all AI generated test cases



```
1 Task-1:Test-Driven Development for Even/Odd Number Validator
2 #Generate test cases for a function is_even(n) and then implement
3 #Example Test Scenarios:a.is_even(2) → True b.is_even(7) → False
4 def is_even(n):
5     """
6     Function to check if a number is even.
7     Input: n (integer)
8     Output: True if n is even, False otherwise
9     """
10    if n % 2 == 0:
11        return True
12    else:
13        return False
14    # Test cases
15    print(is_even(2)) # Expected: True
16    print(is_even(7)) # Expected: False
17    print(is_even(0)) # Expected: True
18    print(is_even(-4)) # Expected: True
19    print(is_even(9)) # Expected: False

PS C:\Users\HP\Desktop\AI> & "C:\Program Files\Python313\python.exe" "c:/Users/HP/Desktop/AI/AI_ASS-8.2(1918).py"
True
False
True
True
False
```

Task 2 Test-Driven Development for String Case Converter

• Ask AI to generate test cases for two functions:

• `to_uppercase(text)`

• `to_lowercase(text)`

Requirements:

- Handle empty strings
- Handle mixed-case input
- Handle invalid inputs such as numbers or None

Example Test Scenarios:

`to_uppercase("ai coding") → "AI CODING"`

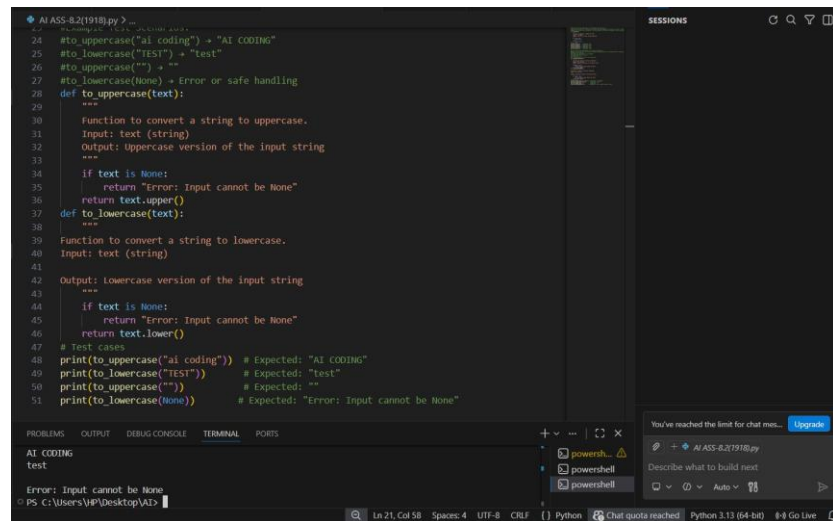
`to_lowercase("TEST") → "test"`

`to_uppercase("") → ""`

to_lowercase(None) → Error or safe handling

Expected Output 2

- Two string conversion functions that pass all AI-generated test cases with safe input handling.



```
AI_ASS-8.2(1918).py
24 #to_uppercase("ai coding") → "AI CODING"
25 #to_lowercase("TEST") → "test"
26 #to_uppercase("") → ""
27 #to_lowercase(None) → Error or safe handling
28 def to_uppercase(text):
29     """
30     Function to convert a string to uppercase.
31     Input: text (string)
32     Output: Uppercase version of the input string
33     """
34     if text is None:
35         return "Error: Input cannot be None"
36     return text.upper()
37 def to_lowercase(text):
38     """
39     Function to convert a string to lowercase.
40     Input: text (string)
41     Output: Lowercase version of the input string
42     """
43     if text is None:
44         return "Error: Input cannot be None"
45     return text.lower()
46 # Test cases
47 print(to_uppercase("ai coding")) # Expected: "AI CODING"
48 print(to_lowercase("TEST")) # Expected: "test"
49 print(to_uppercase("")) # Expected: ""
50 print(to_lowercase(None)) # Expected: "Error: Input cannot be None"

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
AI CODING
test
Error: Input cannot be None
PS C:\Users\VP\Desktop\AI>
```

Task 3 Test-Driven Development for List Sum Calculator

- Use AI to generate test cases for a function sum_list(numbers) that calculates the sum of list elements.

Requirements:

- Handle empty lists
- Handle negative numbers
- Ignore or safely handle non-numeric values

Example Test Scenarios:

sum_list([1, 2, 3] → 6

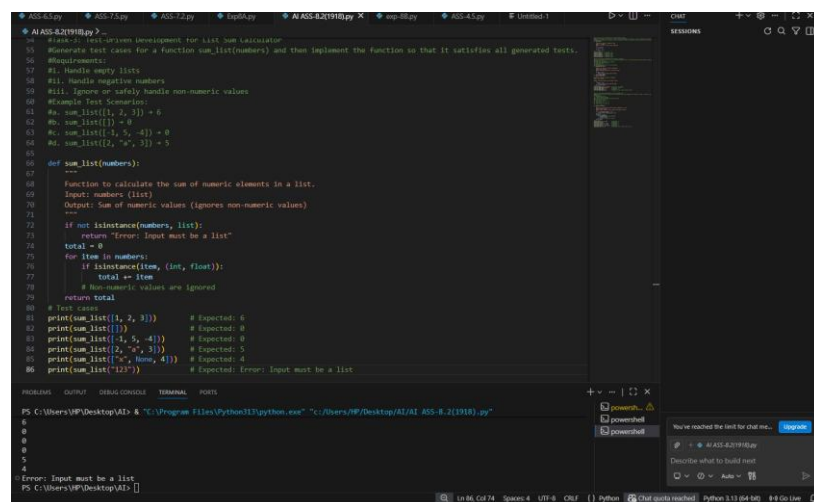
sum_list([]) → 0

sum_list [1, 5, 4] → 10

sum_list([2, "a", 3] → 5

Expected Output 3

- A robust list-sum function validated using AI-generated test cases.



```
AI_ASS-8.2(1918).py
55 #Task 3: Test-Driven Development for List Sum Calculator
56 #Generate test cases for a function sum_list(numbers) and then implement the function so that it satisfies all generated tests.
57 #Requirements:
58 #I. Handle empty lists
59 #II. Handle negative numbers
60 #III. Ignore or safely handle non-numeric values
61 #Example Test Scenarios:
62 #sum_list([1, 2, 3]) → 6
63 #sum_list([]) → 0
64 #sum_list([1, 5, -4]) → 2
65 #sum_list([2, "a", 3]) → 5
66 #sum_list([2, "a", 3]) → 5
67
68 def sum_list(numbers):
69     """
70     Function to calculate the sum of numeric elements in a list.
71     Input: numbers (list)
72     Output: Sum of numeric values (ignores non-numeric values)
73     """
74     if not isinstance(numbers, list):
75         return "Error: Input must be a list"
76     total = 0
77     for item in numbers:
78         if isinstance(item, (int, float)):
79             total += item
80         # Non-numeric values are ignored
81     return total
82
83 # Test cases
84 print(sum_list([1, 2, 3])) # Expected: 6
85 print(sum_list([])) # Expected: 0
86 print(sum_list([1, 5, -4])) # Expected: 2
87 print(sum_list([2, "a", 3])) # Expected: 5
88 print(sum_list([2, "a", 3])) # Expected: 5
89 print(sum_list("123")) # Expected: Error: Input must be a list

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\VP\Desktop\AI> . "C:\Program Files\Python311\python.exe" "C:\Users\VP\Desktop\AI\AI_ASS-8.2(1918).py"
6
0
2
5
5
Error: Input must be a list
PS C:\Users\VP\Desktop\AI>
```

Task 4 Test Cases for Student Result Class

• Generate test cases for a StudentResult class with the following methods:

- add_marks(mark)
- calculate_average()
- get_result()

Requirements:

- Marks must be between 0 and 100
- Average $\geq 40 \rightarrow$ Pass, otherwise Fail

Example Test Scenarios:

Marks: 60, 70, 80 $\rightarrow$ Average: 70 $\rightarrow$ Result: Pass

Marks: 30, 35, 40 $\rightarrow$ Average: 35 $\rightarrow$ Result: Fail

Marks: 10 $\rightarrow$ Error

Expected Output 4

- A fully functional StudentResult class that passes all AI generated test

```
AI ASSIST (Python)
#Task-4: Test Cases for StudentResult Class
#Generate test cases for a StudentResult class with methods:
#1. add_marks(mark)
#2. calculate_average()
#3. get_result()
#
#Requirements:
#1. Marks must be between 0 and 100
#2. Average >= 40 -> Pass, otherwise Fail
#
#Example Test Scenarios:
#1. Marks: [60, 70, 80] -> Average: 70 -> Result: Pass
#2. Marks: [30, 35, 40] -> Average: 35 -> Result: Fail
#3. Marks: [10] -> Error

class StudentResult:
    """
    Class to store student marks and calculate result.
    """
    def __init__(self):
        self.marks = []
    def add_marks(self, mark):
        """
        Adds a mark to the list.
        Mark must be between 0 and 100.
        """
        if not isinstance(mark, (int, float)):
            return "Error: Marks must be numeric"
        if mark < 0 or mark > 100:
            return "Error: Marks must be between 0 and 100"
        self.marks.append(mark)
    def calculate_average(self):
        """
        Calculates average of marks.
        """
        return 0
        if len(self.marks) == 0:
            return 0

PS C:\Users\VP\Desktop\AI> & "C:\Program Files\Python311\python.exe" "c:/Users/VP/Desktop/AI/AI_ASS-8.2(1918).py"
70.0
Pass
35.0
```

```
AI ASSIST (Python)
class StudentResult:
    def calculate_average(self):
        return 0
    def get_result(self):
        """
        Returns Pass if average >= 40 else Fail.
        """
        avg = self.calculate_average()
        if avg >= 40:
            return "Pass"
        else:
            return "Fail"
# Test Cases
# Test Case 1: Pass Scenario
student1 = StudentResult()
student1.add_marks(60)
student1.add_marks(70)
student1.add_marks(80)
print(student1.calculate_average()) # Expected: 70.0
print(student1.get_result()) # Expected: Pass
# Test Case 2: Fail Scenario
student2 = StudentResult()
student2.add_marks(30)
student2.add_marks(35)
student2.add_marks(40)
print(student2.calculate_average()) # Expected: 35.0
print(student2.get_result()) # Expected: Fail
# Test Case 3: Invalid Marks
student3 = StudentResult()
print(student3.add_marks(-10)) # Expected: Error
print(student3.add_marks(120)) # Expected: Error
# Test Case 4: Empty Marks
student4 = StudentResult()
print(student4.calculate_average()) # Expected: 0
print(student4.get_result()) # Expected: Fail
```

```
PS C:\Users\HP\Desktop\AI> & "C:\Program Files\Python313\python.exe" "C:\Users\HP\Desktop\AI\AI_Ass_8-2(1918).py"
70.0
Pass
35.0
Fail
Error: Marks must be between 0 and 100
70.0
Pass
35.0
Fail
Error: Marks must be between 0 and 100
Error: Marks must be between 0 and 100
Error: Marks must be between 0 and 100
0
Fail
PS C:\Users\HP\Desktop\AI>
```

Task 5 Test-Driven Development for Username Validator

Requirements:

- Minimum length: 5 characters
- No spaces allowed
- Only alphanumeric characters

Example Test Scenarios:

is_valid_username("user01") → True

is_valid_username("ai") → False

is_valid_username("user name") → False

is_valid_username("user@123") → False

Expected Output 5

A username validation function that passes all AI-generated test cases.

```
AI ASS-8.2(1918).py > ...
159 #Task-5: Test-Driven Development for Username Validator
160 #Requirements:
161 # - Minimum 5 characters
162 # - No spaces
163 # - Only alphanumeric characters
164 def is_valid_username(username):
165     if not isinstance(username, str):
166         return False
167     if len(username) < 5:
168         return False
169     if " " in username:
170         return False
171     if not username.isalnum():
172         return False
173     return True
174 # Test Cases
175 print(is_valid_username("user01")) # Expected: True
176 print(is_valid_username("ai")) # Expected: False
177 print(is_valid_username("user name")) # Expected: False
178 print(is_valid_username("user@123")) # Expected: False
179 print(is_valid_username("12345")) # Expected: True
180 print(is_valid_username("")) # Expected: False
181 print(is_valid_username(None)) # Expected: False

PS C:\Users\HP\Desktop\AI> & "C:\Program Files\Python313\python.exe" "C:\Users\HP\Desktop\AI\AI_Ass-8-2(1918).py"
True
False
False
True
False
False
False
PS C:\Users\HP\Desktop\AI>
```

DOCTEST

python -m doctest -v Ass_8_2.py

True

False

True

True

False

AI CODING

test

Error: Input cannot be None

6

0

0

```

5
Average: 70.0
Result: Pass
Average: 35.0
Result: Fail
True
False
False
False
All tests passed successfully.
Trying:
is_even(2)
Expecting:
True
ok
Trying:
is_even(7)
Expecting:
False
ok
Trying:
is_even(0)
Expecting:
True
ok
Trying:
is_even(-4)
Expecting:
True
ok
Trying:
is_even(9)
Expecting:
False
ok
10 items had no tests:
Ass_8_2
Ass_8_2.StudentResult
Ass_8_2.StudentResult.init
Ass_8_2.StudentResult.add_marks
Ass_8_2.StudentResult.calculate_average
Ass_8_2.StudentResult.get_result
Ass_8_2.is_valid_username
Ass_8_2.sum_list
Ass_8_2.to_lowercase
Ass_8_2.to_uppercase
1 items passed all tests:
5 tests in Ass_8_2.is_even
5 tests in 11 items.
5 passed and 0 failed.
Test passed.

import pytest
from Ass_8_2 import is_even, to_uppercase, to_lowercase, sum_list, StudentResult, is_vali
d_username

```

Test cases for Task 1 - Even/Odd Number Validator

```

def test_is_even():
    assert is_even(2) == True
    assert is_even(7) == False
    assert is_even(0) == True

```

```
assert is_even(-4) == True
assert is_even(9) == False
```

Test cases for Task 2 - String Case Converter

```
def test_to_uppercase():
    assert to_uppercase("ai coding") == "AI CODING"
    assert to_uppercase("") == ""
    assert to_uppercase(None) == "Error: Input cannot be None"
    assert to_uppercase("Test") == "TEST"
def test_to_lowercase():
    assert to_lowercase("TEST") == "test"
    assert to_lowercase("") == ""
    assert to_lowercase(None) == "Error: Input cannot be None"
    assert to_lowercase("Test") == "test"
```

Test cases for Task 3 - List Sum Calculator

```
def test_sum_list():
    assert sum_list([1, 2, 3]) == 6
    assert sum_list([]) == 0
    assert sum_list([1, 5, 4]) == 10
    assert sum_list([2, "a", 3]) == 5
    assert sum_list([2, 3, "a", 4]) == 6
    assert sum_list([100, 50, 20]) == 170
```

Test cases for Task 4 - StudentResult Class

```
def test_student_result():
    student = StudentResult()
    student.add_marks(60)
    student.add_marks(70)
    student.add_marks(80)
    assert student.calculate_average() == 70.0
    assert student.get_result() == "Pass"
    student = StudentResult()
    student.add_marks(30)
    student.add_marks(35)
    student.add_marks(40)
    assert student.calculate_average() == 35.0
    assert student.get_result() == "Fail"
```

Test cases for Task 5 - Username Validator

```
def test_is_valid_username():
    assert is_valid_username("user01") == True
    assert is_valid_username("ai") == False
    assert is_valid_username("user name") == False
    assert is_valid_username("user@123") == False
    assert is_valid_username("validUser") == True
    assert is_valid_username("us") == False
```

platform win32 -- Python 3.12.3, pytest-9.0.2, pluggy-1.6.0 rootdir: PS
C:\Users\HP\Desktop\AI>

A.I.AC

rootdir: PS C:\Users\HP\Desktop\AI> A.I.AC

collected 6 items

Ass_8_2.py

[100%]

=====

6 passed in 0.09s